

# MINDORA

## IA Educativa Offline

---

Aplicación de escritorio para uso estudiantil gratuito.  
Sube apuntes, genera exámenes, pregunta y estudia.  
Todo funciona **sin internet** una vez instalada.

Desarrollado por **Endika Pradera Touzani** · Marzo 2026

---

## MINDORA — IA Educativa Offline ( EN DESARROLLO )

*Aplicación de escritorio para USO ESTUDIANTIL GRATUITO: sube apuntes, genera exámenes, responde preguntas y tutoriza alumnos. Todo funciona **sin internet** una vez instalada.*

---

### Últimas incorporaciones

#### Chat de estudio renovado + guardado de conversación (Mar 2026)

**Problema:** la sección de estudio mezclaba controles de estilos y botones que no siempre aportaban valor en el flujo principal. La experiencia no se sentía como un chat natural.

**Qué se hizo:**

- Se simplificó la interacción a formato **chat normal** (mensaje → respuesta del tutor).

- Se eliminaron controles antiguos de estilo en la vista principal de estudio.
- Se añadió botón **Guardar conversación** para exportar el historial a [.txt](#).
- Se rediseñó la interfaz del chat con layout moderno (historial lateral + hilo principal + composer compacto).

Resultado: flujo más limpio, más rápido para estudiar y sin perder conversaciones importantes.

---

## Reranker híbrido para mejorar precisión (Mar 2026)

**Problema:** con apuntes densos o ruidosos, el recuperador podía priorizar fragmentos semánticamente cercanos pero didácticamente peores para responder una duda concreta.

**Qué se hizo:**

- Se añadió **reranking híbrido** (similitud semántica + señales léxicas + intención de la pregunta + metadatos).
- Se añadió selección con **diversidad** para evitar exceso de fragmentos del mismo documento/tema.
- Se reforzó la clasificación por tipo de contenido (definición, ejemplo, tabla, legal) para elegir mejor contexto.

Resultado: menos mezcla de temas, mejores fuentes recuperadas y respuestas más fiables.

---

## Generación de examen más robusta (Mar 2026)

**Problema:** en ciertos casos el LLM podía devolver preguntas repetidas o no llegar al número solicitado.

**Qué se hizo:**

- Dedupe de preguntas por enunciado normalizado.
- Pase de completado para generar solo las preguntas faltantes cuando el primer intento no llega al total.
- Métricas de calidad visibles en UI (confianza media + alertas de distractores).

Resultado: generación de examen más consistente y útil para práctica real.

---

## Subida de documentos con progreso real (Mar 2026)

**Problema:** el usuario no tenía feedback fiable del avance durante la subida.

**Qué se hizo:**

- Se implementó progreso real de upload (evento [progress](#)) con barra visual.
- Se bloquean controles durante subida para evitar errores de interacción.

Resultado: experiencia de ingesta más clara y profesional.

---

## Hotfix — App colgada al arrancar (Mar 2026)

**Problema:** al abrir MINDORA, la pantalla de carga se quedaba bloqueada indefinidamente y la app nunca llegaba a mostrar la interfaz principal.

**Por qué ocurría:** el componente principal ([App.tsx](#)) lanza al arrancar un bucle de *\*health polling\** que envía una petición a [/health](#) cada 1.2 s para saber si el backend Python ya está listo. Ese bucle tenía los bloques [try-catch](#) mal anidados: el [catch](#) estaba colocado dentro del [if](#) en lugar de fuera, rompiendo el flujo de control.

El bucle nunca salía, la UI nunca recibía la señal de "backend listo" y el usuario veía la rueda girando para siempre.

**Cómo se resolvió:** se reescribió completamente la función `poll()` con la estructura correcta. Ahora tiene un límite explícito de 50 intentos (~60 s máximo). Si el backend responde, carga automáticamente el estado del modelo LLM y las ramas disponibles. Si agota los intentos sin respuesta, muestra un error claro en pantalla en lugar de quedarse bloqueado.

---

## Banco de tests — 108 pruebas automatizadas (Mar 2026)

**Por qué:** una aplicación que genera exámenes y evalúa alumnos necesita garantías de que cada pieza funciona correctamente antes de llegar al usuario. Sin tests automatizados, cualquier cambio puede romper silenciosamente la generación de exámenes, la subida de documentos o el ciclo completo de simulacro sin que nadie se entere hasta que el usuario lo sufre.

**Qué se hizo:** se crearon tres temarios educativos reales en `core/tests/fixtures/` (matemáticas, programación y física) para que los tests trabajen con contenido auténtico y no con texto artificial. Sobre esos fixtures se construyó un banco de 50 tests de integración en `core/tests/test_integration.py`:

- `TestBranchCRUD` (9 tests): verifica que se pueden crear ramas por asignatura, listarlas y eliminarlas, y que los errores —nombre vacío, rama duplicada, rama inexistente— se rechazan correctamente con el código HTTP adecuado.
  - `TestTextExtraction` (8 tests): comprueba que el extractor de texto parte los tres temarios en fragmentos (\*chunks\*) del tamaño correcto sin perder contenido ni generar fragmentos vacíos.
  - `TestDocumentIngest` (5 tests): valida que un documento sube correctamente, se generan sus embeddings y se indexa en FAISS para que la búsqueda semántica posterior funcione sobre él.
  - `TestExamGeneration` (6 tests): asegura que el generador produce exámenes con el número exacto de preguntas pedido, en el formato correcto (test, desarrollo o mixto) y respetando el nivel de dificultad indicado.
  - `TestExamValidator` (6 tests): comprueba el validador de distractores —que las respuestas incorrectas no sean demasiado obvias ni idénticas a la correcta— y el \*confidence score\* asignado a cada pregunta.
  - `TestFullLifecycle` (1 test): ejecuta el ciclo completo end-to-end: crea una rama → sube un documento → genera un examen → lanza un simulacro → borra la rama. Si este test pasa, la app funciona de principio a fin.
  - `TestErrorValidation` (6 tests): garantiza que todos los endpoints rechazan inputs inválidos con los códigos HTTP y mensajes de error correctos, sin lanzar excepciones no controladas.
  - `TestBancoPreguntas` (9 tests): banco de preguntas concretas sobre los tres temarios para verificar que el sistema RAG recupera el contexto correcto y genera respuestas coherentes con el contenido subido.
- 

## Validación inline en formularios (Mar 2026)

**Problema:** cuando el usuario cometía un error —nombre de rama vacío, archivo no seleccionado, pregunta demasiado corta— aparecía un \*badge\* de estado genérico en otro punto de la pantalla. El usuario no sabía exactamente qué campo estaba mal y el feedback llegaba tarde.

**Qué se hizo:** se eliminaron los badges de error genéricos y se implementó validación inline directamente bajo cada campo del formulario. Cada input tiene su propio estado de error (`branchNameError`, `fileError`, `questionError`, `examTopicError`, `learnPhraseError`) que aparece en rojo justo debajo del campo afectado en el momento en que el usuario intenta enviar. Para las confirmaciones de éxito y los errores de operación (subida completada, rama eliminada, fallo de red...) se añadió un sistema de **toasts**: notificaciones emergentes que aparecen 3.5 s y desaparecen solas, sin ocupar espacio permanente en la interfaz ni exigir que el usuario las cierre manualmente.

---

## Fase 5 — Fine-tuning LoRA (Feb 2026)

**Por qué:** el modelo base (Mistral 7B) responde bien en general, pero no conoce el estilo pedagógico concreto de MINDORA. Con fine-tuning LoRA se puede especializar el modelo con ejemplos reales de buenas respuestas aprobadas por el usuario, mejorando la calidad de forma progresiva sin necesidad de reentrenar los 7.000 millones de parámetros del modelo base.

**Qué hace:** se construyó un pipeline que recoge los pares pregunta-respuesta que el usuario ha valorado positivamente, los convierte en un dataset JSONL compatible con LoRA, y los usa para ajustar un adaptador ligero (pocos MB) que se superpone al modelo base. El script [train\\_lora.py](#) gestiona el entrenamiento localmente. Al arrancar la app, el backend detecta automáticamente si existe un adaptador entrenado y lo carga sobre el modelo base, indicando en la UI si el modo fine-tuning está activo o no.

---

## Fase 3 — OCR automático en PDFs (Feb 2026)

**Por qué:** muchos apuntes universitarios son PDFs escaneados —fotos de páginas físicas— y no PDFs con texto real incrustado. Sin OCR, esos documentos se subirían sin error pero el sistema no extraería ningún texto, haciéndolos completamente inútiles para el chat y los exámenes.

**Qué hace:** al ingestar un PDF, el sistema detecta automáticamente si las páginas contienen texto real o son imágenes escaneadas. En el segundo caso, usa **PyMuPDF** para renderizar cada página como imagen y **Tesseract** para extraer el texto mediante reconocimiento óptico de caracteres. Además extrae las imágenes embebidas en documentos mixtos. El resultado es siempre el mismo: texto limpio listo para embeddings y RAG, independientemente de si el PDF era nativo o escaneado.

---

## Fase 2 — Calidad y personalización de respuestas (Feb 2026, base)

**Por qué:** las respuestas del modelo base suenan a menudo robóticas, excesivamente formales o innecesariamente largas. Un alumno necesita respuestas adaptadas a cómo está estudiando: a veces quiere un resumen rápido, otras un paso a paso detallado, otras que se lo expliquen como un compañero.

**Qué hace:** se construyó la base de estilo docente (plantillas y \*few-shot\*) en [llm.py](#) y postproceso anti-respuesta robótica. Actualmente, la experiencia principal de estudio se presenta en formato **chat natural con tutor-profesor**, manteniendo aprendizaje por feedback útil/no útil y diccionario de intenciones para personalización progresiva.

---

## Fase 1 — RAG estructurado con fuentes citadas (Ene 2026)

**Por qué:** sin RAG (\*Retrieval-Augmented Generation\*), el modelo solo puede responder desde su conocimiento general de entrenamiento, sin acceso al contenido específico de los apuntes del alumno. El resultado serían respuestas genéricas que no reflejan lo que el profesor explicó ni los conceptos del temario concreto.

**Qué hace:** cuando el usuario hace una pregunta, el sistema la convierte en un vector de embeddings usando [all-MiniLM-L6-v2](#) —modelo local, sin necesidad de internet— y busca en el índice FAISS los fragmentos de los apuntes subidos que más se parecen semánticamente a la pregunta. Esos fragmentos se inyectan como contexto en el prompt del modelo, que genera una respuesta basada en ellos e indica las fuentes (documento y página) de donde viene la información. Para los exámenes, se añadió un validador que comprueba que los distractores sean plausibles pero claramente distintos de la correcta, y asigna un \*confidence score\* a cada pregunta para que el sistema descarte automáticamente las de baja calidad.

---

## ■ ■ Descargar e instalar (usuarios finales)

| Plataforma            | Descarga                        | Requisitos        |
|-----------------------|---------------------------------|-------------------|
| macOS (Apple Silicon) | <a href="#">Releases → .dmg</a> | macOS 12+         |
| Windows (x64)         | <a href="#">Releases → .exe</a> | Windows 10/11 x64 |

## ■ ■ Modelo LLM — descarga obligatoria (una sola vez, ~4.1 GB)

Antes de arrancar la app por primera vez, coloca este archivo en tu carpeta de Documentos:

- **macOS:** `~/Documents/MINDORA/models/mistral-7b-instruct-v0.2.Q4KM.gguf`
- **Windows:** `C:\Users\<tu nombre>\Documents\MINDORA\models\mistral-7b-instruct-v0.2.Q4KM.gguf`

Descárgalo de: [HuggingFace — Mistral 7B Instruct Q4KM](#)

## Publicar una nueva versión (solo para el desarrollador)

### 1. Obtener el instalador de Windows

1. Ve a **Actions** → **Build Windows Installer**
2. Espera a que el workflow termine con ■ (~20-30 min la primera vez)
3. Entra al workflow completado → sección **Artifacts** → descarga [MINDORA-Windows-xxxx.exe](#)

### 2. Publicar el Release en GitHub

1. Ve a **Releases** → **Create a new release**
2. Pon el tag de versión: `v1.0.0`
3. Escribe el título: `MINDORA v1.0.0`
4. Adjunta los dos archivos:
  - [MINDORA1.0.0aarch64.dmg](#) (macOS — generado con `./build.sh` en tu Mac)
  - [MINDORA1.0.0x64-setup.exe](#) (Windows — descargado de Actions en el paso anterior)
5. Publica el release

Desde ese momento cualquier usuario puede ir a la página de Releases y descargar el instalador de su plataforma.

## Compilar desde código fuente

### macOS (Apple Silicon / Intel)

```
# Requisitos: Python 3.9+, Node 18+, Rust (rustup.rs)
chmod +x build.sh
./build.sh
# → Genera ui/src-tauri/target/release/bundle/dmg/MINDORA_1.0.0_aarch64.dmg
```

## Windows — vía GitHub Actions (recomendado)

La forma más sencilla es dejar que GitHub compile por ti:

1. Haz un push a [main](#)
2. Ve a **Actions** → **Build Windows Installer**
3. Descarga el artefacto [MINDORA-Windows-\\*.exe](#) cuando el workflow termine (~20-30 min)

Workflow: [.github/workflows/build-windows.yml](#)

## Windows — manual (desde una máquina Windows)

```
# Requisitos: Python 3.11, Node 18+, Rust, VS Build Tools 2022
cd core
pip install -r requirements.txt pyinstaller
python -m PyInstaller pyinstaller_windows.spec --clean --noconfirm
cd ../ui
npm install
npm run tauri build
# → ui/src-tauri/target/release/bundle/nsis/MINDORA_1.0.0_x64-setup.exe
```

## Estructura del proyecto

|                                 |                                       |
|---------------------------------|---------------------------------------|
| MINDORA/                        |                                       |
| ■■■■ build.sh                   | ← Build completo (macOS)              |
| ■■■■ .github/workflows/         |                                       |
| ■ ■■■■ build-windows.yml        | ← GitHub Actions: compila Windows     |
| ■■■■ core/                      | ← Backend Python (FastAPI)            |
| ■ ■■■■ app/                     |                                       |
| ■ ■ ■■■■ api/routes/            | ← Endpoints REST                      |
| ■ ■ ■■■■ services/              | ← LLM, embeddings, exámenes           |
| ■ ■ ■■■■ storage/               | ← Ramas, documentos, FAISS            |
| ■ ■■■■ pyinstaller.spec         | ← Spec PyInstaller macOS              |
| ■ ■■■■ pyinstaller_windows.spec | ← Spec PyInstaller Windows            |
| ■ ■■■■ requirements.txt         |                                       |
| ■ ■■■■ run_server.py            |                                       |
| ■■■■ ui/                        | ← Frontend Tauri + React + TypeScript |
| ■■■■ src/App.tsx                | ← Componente principal                |
| ■■■■ src/api.ts                 | ← Llamadas al backend                 |
| ■■■■ src-tauri/                 |                                       |
| ■■■■ src/main.rs                | ← Rust: lanza backend                 |
| ■■■■ tauri.conf.json            |                                       |
| ■■■■ icons/                     | ← Iconos (todas las plataformas)      |

## Características

- **Ramas (temarios):** crea ramas por asignatura, sube PDFs, Word, PowerPoint
- **Chat IA tipo profesor:** conversación natural sobre tus apuntes, sin internet
- **Guardar conversación:** exporta el historial de chat a `.txt`
- **Generador de exámenes:** tipo test, desarrollo o mixto, ajustable por dificultad
- **Búsqueda semántica + reranker:** FAISS + embeddings locales + reordenado híbrido de contexto
- **Asistente:** conversación libre sobre el contenido subido
- **Gestión de ramas:** crea, selecciona y elimina con confirmación

## Stack tecnológico

| Capa          | Tecnología                                           |
|---------------|------------------------------------------------------|
| Frontend      | React 18 + TypeScript + Vite                         |
| Desktop shell | Tauri v1 (Rust)                                      |
| Backend API   | FastAPI + Uvicorn                                    |
| LLM local     | llama-cpp-python (Mistral 7B Q4)                     |
| Embeddings    | sentence-transformers (multilingüe + fallback local) |
| Vector store  | FAISS                                                |
| Empaquetado   | PyInstaller + NSIS (Windows) / hdiutil (macOS)       |

## Idea de negocio: por qué MINDORA tiene potencial

MINDORA nace para resolver un problema real: estudiar con IA sin exponer datos personales ni depender de internet.

### Propuesta de valor

- **100% offline:** los apuntes y preguntas no salen del equipo del usuario.
- **Privacidad por diseño:** ideal para estudiantes, centros educativos y organizaciones con datos sensibles.
- **Especialización educativa:** no es un chat generalista; está centrada en temarios, estudio y evaluación.
- **Flujo completo en una sola app:** ingesta de documentos, RAG, simulacros y generación de exámenes.

### Por qué es una oportunidad

- Las plataformas grandes priorizan la nube y el uso masivo.
- MINDORA se posiciona en un nicho creciente: **IA local aplicada a educación**.
- La tecnología reciente (GGUF, cuantización, llama.cpp, embeddings locales) permite ofrecer buena experiencia en hardware de usuario.

### Ventaja competitiva

MINDORA combina funcionalidades que normalmente están separadas:

1. Tutor IA sobre documentos propios.
2. Generación de exámenes y solucionarios.
3. Simulacros evaluables con histórico de progreso.
4. Todo funcionando en local y con control de datos.

En resumen: puede posicionarse como **“asistente académico offline”** para estudiantes, opositores, academias y universidades.

---

## Licencia

---

Consulta el archivo [LICENSE](#) para más información.